

리스트와 연결 리스트 정리 (강의 기반 + 시각 자료)

리스트란?

- **리스트(List)** 또는 **선형 리스트(Linear List)**: 순서가 있는 항목들의 집합
- **일상 속 예시**
 - 요일 목록: 월, 화, 수, 목, 금, 토, 일
 - 자음 목록: ㄱ, ㄴ, ㄷ, ㄹ...
 - 게임 카드 목록, 할 일 목록 등

리스트 주요 연산

연산 종류	설명
삽입	항목을 추가 (처음/중간/끝 위치 가능)
삭제	특정 위치 또는 전체 항목 제거
검색	특정 항목이 존재하는지 확인
조회	특정 위치의 항목 반환
개수 확인	리스트에 포함된 항목 수 세기
상태 확인	비어있는지, 가득 찼는지 확인
출력	리스트 전체 내용 표시

```
void insert(List *list, int position, Element item);
```

리스트 구현 방법

배열 기반 구현 (Sequential Representation)

도식 예시

```
[10] → [20] → [30] → [40] → NULL
```

- 메모리에 **연속적**으로 저장됨
- 삽입/삭제 시 요소 **이동** 필요 → 오버헤드 발생
- **정해진 크기**로 선언됨 → 크기 제한 있음

연결 리스트 구현 (Linked Representation)

도식 예시

```
[Data:10|Next] → [Data:20|Next] → [Data:30|Next] → NULL
```

- 각 노드는 **데이터 + 포인터**로 구성됨
- 메모리 **분산적**으로 동적 할당
- 크기 제한 없음

```
typedef struct Node {  
    Element data;  
    struct Node* next;  
} Node;
```

```
newNode->link = prev->link;  
prev->link = newNode;
```

🔗 연결 리스트의 유형

종류	설명
단순 연결 리스트	한 방향 연결, 마지막은 NULL
원형 연결 리스트	마지막 노드가 첫 노드를 가리킴
이중 연결 리스트	양방향 포인터 사용
환형 이중 리스트	이중 연결 + 원형 구조

구조 도식

단순 연결 리스트:

A → B → C → NULL

원형 연결 리스트:

A → B → C ↴

↑ ↓

←←←←←←←←←←

이중 연결 리스트:

NULL ← A ⇄ B ⇄ C → NULL

환형 이중 리스트:

A ⇄ B ⇄ C

↑ ↓
 ←←←←←←←←←←←←←←←←

단순 연결 리스트 삽입 & 삭제

```
Node* insertFirst(Node* head, int value);
Node* insertAfter(Node* prevNode, int value);

Node* deleteFirst(Node* head);
Node* deleteAfter(Node* prev);
```

삽입 전/후 도식

Before: A → B → C
 Insert X after A
 After : A → X → B → C

삭제 전/후 도식

Before: A → B → C
 Delete B
 After : A → C

탐색 연산

```
Node* search(Node* head, int value) {
    Node* p = head;
    while (p != NULL && p->data != value)
        p = p->link;
    return p;
}
```

리스트 뒤집기 (역순)

```
Node* reverse(Node* head) {
    Node* prev = NULL;
    Node* curr = head;
    Node* next;
```

```

    while (curr != NULL) {
        next = curr->link;
        curr->link = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}

```

병합 연산

```

Node* merge(Node* head1, Node* head2) {
    if (head1 == NULL) return head2;
    if (head2 == NULL) return head1;
    Node* p = head1;
    while (p->link != NULL)
        p = p->link;
    p->link = head2;
    return head1;
}

```

실습: 문자열 저장 리스트

```

typedef struct Node {
    char data[20];
    struct Node* link;
} Node;

void insertFirst(Node** head, const char* str) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    strcpy(newNode->data, str);
    newNode->link = *head;
    *head = newNode;
}

```

입력

```

insertFirst(&head, "Apple");
insertFirst(&head, "Kiwi");
insertFirst(&head, "Banana");

```

출력 도식

```
[Banana] → [Kiwi] → [Apple] → NULL
```

🎓 최종 비교 정리

항목	배열 기반 리스트	연결 리스트
구현 난이도	쉬움	복잡
삽입/삭제 성능	느림 (이동 필요)	빠름 (포인터 조작)
메모리 사용	고정 크기	동적 크기
메모리 구조	연속적	분산적 (동적 할당)

☑ 연결 리스트는 삽입/삭제에 유연하며, 동적 구조에서 강력한 도구입니다.